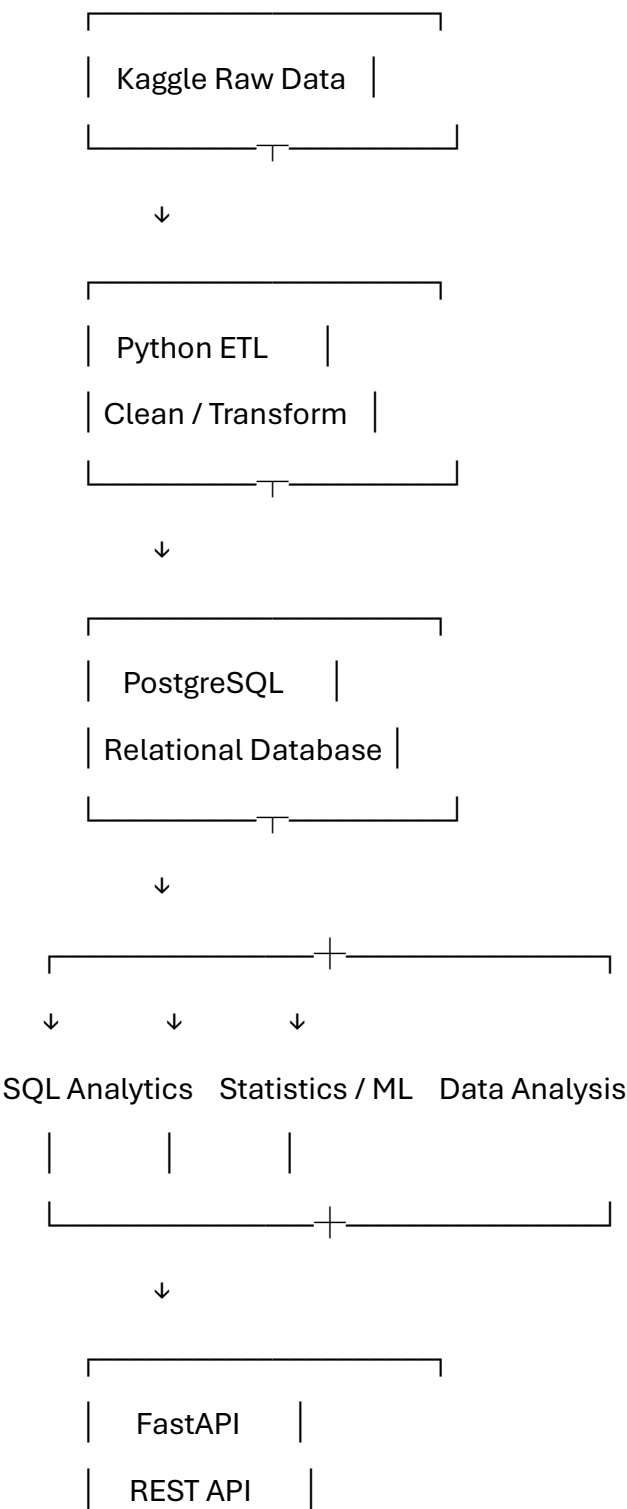
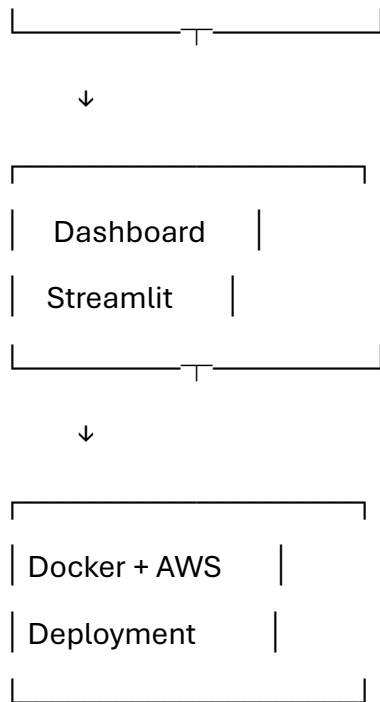


Movie Intelligence Platform — Complete Project Plan

Target architecture





Stage 0 — Define the Project

Goal

Decide exactly what the platform is supposed to answer.

Main work

Define 4 major areas:

1. Movie economics

- Budget
- Revenue
- Profit
- ROI

2. Audience behaviour

- Ratings
- Rating distributions
- Popularity

- Rating count

3. Movie/industry analysis

- Genres
- Directors
- Cast
- Release periods
- Languages

4. Prediction / recommendation

- Revenue prediction
- Movie recommendation

Core decisions

Decide that the project is:

An analytical platform for understanding movies and audience behaviour, with predictive and recommendation capabilities.

Not:

“A machine-learning project about movies.”

Stack

- Python
- Git/GitHub
- Jupyter initially

Milestone 0

You have:

- Project scope written
- Main analytical questions defined
- Repository created
- README started

- Raw dataset downloaded and preserved
-

Stage 1 — Understand the Raw Data

Goal

Understand what you actually have before writing serious code.

Main work

Inspect:

- movies_metadata.csv
- credits.csv
- keywords.csv
- links.csv
- ratings.csv

Determine:

- row counts
- columns
- data types
- missing values
- duplicates
- IDs
- relationships between datasets
- malformed records
- JSON/stringified fields

Build a **data dictionary**.

Core decisions

Determine:

- Which column is the movie identifier?

- Which tables are one-to-many?
- Which records cannot be reliably joined?
- Which fields are trustworthy?
- Which fields need transformation?

Stack

- Python
- pandas
- Jupyter

Milestone 1 — Data Understanding

You should be able to explain **every dataset and its relationships without looking at the code**.

You should have:

- Data dictionary
- Data-quality report
- Relationship diagram
- List of cleaning decisions
- Clear understanding of IDs and joins

Stage 2 — Build the ETL Pipeline

Goal

Turn the messy Kaggle files into clean, reproducible data.

Main work

Build:

Raw CSV

↓

Extract

↓

Clean

↓

Transform

↓

Validate

↓

Load

Handle:

- missing values
- duplicate movies
- malformed dates
- numeric conversion
- JSON parsing
- genres
- cast
- crew
- keywords
- ID inconsistencies

Make the pipeline **re-runnable**.

Don't manually clean the CSV and save the result.

Core decision

Separate:

raw data

from:

processed data

Never modify the raw data.

Stack

- Python
- pandas
- NumPy
- JSON
- pytest

Milestone 2 — Reproducible ETL

Run one command and produce the processed datasets.

- ETL scripts work from raw data
- Cleaning is reproducible
- Validation checks exist
- Raw data remains untouched
- Running ETL twice produces consistent results

Stage 3 — Design the PostgreSQL Database

Goal

Turn the dataset into a proper relational database.

Main schema

Something approximately like:

movies

genres

movie_genres

people

movie_cast

movie_crew

keywords

movie_keywords

users

ratings

Core decisions

Decide:

- primary keys
- foreign keys
- normalization
- many-to-many relationships
- data types
- constraints
- indexes

Don't just dump CSVs into PostgreSQL.

Stack

- PostgreSQL
- SQL
- pgAdmin/DBeaver
- Python + psycopg/SQLAlchemy

Milestone 3 — Database

You can:

- Explain your schema
- Explain why tables are separated

- Insert/load the entire dataset
 - Enforce important relationships
 - Query the database directly
 - Demonstrate indexes and why they exist
-

Stage 4 — Become Very Strong at SQL Through the Project

Goal

Make SQL one of the strongest parts of the project.

Start with:

SELECT

WHERE

GROUP BY

HAVING

ORDER BY

JOIN

Then:

CTE

CASE

subqueries

window functions

Then:

RANK()

LAG()

LEAD()

PARTITION BY

rolling calculations

Analysis examples

Find:

- highest-grossing movies
- highest ROI movies
- top genres by decade
- directors with strongest performance
- movies whose ratings outperform their genre average
- yearly trends
- rating behaviour by user/movie

Milestone 4 — SQL

You should be able to solve non-trivial analytical questions **without pandas doing the work for you**.

- Complex joins
- CTEs
- Window functions
- Aggregations
- Analytical queries
- Understand query plans at a basic level

Stage 5 — Exploratory & Statistical Analysis

Goal

Move from:

“What does the dataset contain?”

to:

“What can we actually conclude from it?”

Main questions

Examples:

Economics

- Does larger budget correspond to higher revenue?
- What affects ROI?
- Are expensive movies consistently more commercially successful?

Ratings

- Does popularity correspond to ratings?
- How does rating count affect confidence?
- Are highly rated movies necessarily commercially successful?

Genres

- How have genres changed over time?
- Which genres have different economic characteristics?

People

- Do particular directors consistently outperform their genre baseline?

Statistics

Use:

- distributions
- correlation
- confidence intervals
- hypothesis tests
- regression
- effect sizes
- sampling considerations

Core decision

Don't confuse correlation with causation.

Document confounders and limitations.

Stack

- Python
- pandas
- NumPy
- SciPy
- statsmodels
- matplotlib

Milestone 5 — Statistical Analysis

For every major conclusion you can answer:

1. What did I observe?
2. How did I measure it?
3. How uncertain is it?
4. What could explain it?
5. What can I **not** conclude?

Stage 6 — Build the ML Components

Only start this after the analytical foundation is solid.

6A — Revenue Prediction

Goal

Predict movie revenue.

Possible features:

budget

runtime

release year

genres

language

popularity

vote statistics

cast/crew features

Models

Start simple:

1. Linear regression
2. Random Forest
3. Gradient boosting

Compare them properly.

Important decisions

Handle:

- skewed revenue
- outliers
- missing values
- categorical variables
- leakage
- train/test split

Don't just chase the highest metric.

6B — Recommendation System

Build:

Content-based

Use:

- genres
- keywords
- cast

- crew
- movie metadata

Then later:

Collaborative filtering

Use:

user × movie × rating

Milestone 6 — ML

You have:

- Baseline model
- Multiple models compared
- Proper evaluation
- Feature engineering
- Leakage checked
- Error analysis
- Recommendation system
- Clear explanation of limitations

Stage 7 — Build the Analytics Layer

Goal

Stop having analysis scattered across notebooks.

Create reusable Python functions/services.

For example:

analytics/

revenue.py

ratings.py

genres.py

directors.py

recommendations.py

Functions should answer things like:

get_top_movies()

get_genre_statistics()

get_director_performance()

predict_revenue()

recommend_movies()

Core decision

Separate:

data access

business/analytical logic

visualization

Don't put everything into one giant notebook.

Stack

- Python
- PostgreSQL
- SQLAlchemy

Milestone 7

Your analytical functionality can be called from Python without opening a Jupyter notebook.

Stage 8 — Build the API

Goal

Expose your platform through a real application interface.

FastAPI endpoints

Something like:

GET /movies/{id}

GET /movies/{id}/ratings

GET /movies/{id}/similar

GET /movies/top

GET /genres/{genre}

GET /directors/{id}

GET /analytics/revenue

GET /analytics/ratings

POST /predict/revenue

GET /recommendations/{user_id}

Core decisions

Learn:

- request/response models
- validation
- error handling
- database connections
- configuration
- API documentation

Stack

- FastAPI
- Pydantic
- SQLAlchemy

- PostgreSQL
- Unicorn

Milestone 8 — API

You can start the server and successfully:

- Retrieve movie information
 - Query analytical results
 - Get recommendations
 - Make predictions
 - Handle invalid requests
 - Connect API → PostgreSQL correctly
-

Stage 9 — Build the Dashboard

Goal

Give a human-friendly interface to your analytical platform.

Pages

Overview

Movies

Ratings

Revenue

Genres

Movie Explorer

Search → movie → statistics → similar movies

Analytics

Interactive:

- revenue
- ratings

- genres
- directors
- time trends

Prediction

Movie characteristics → predicted revenue

Recommendations

Movie/user → recommended movies

Stack

- Streamlit
- Plotly or matplotlib

Milestone 9

Someone who knows nothing about your code can open the dashboard and actually use the platform.

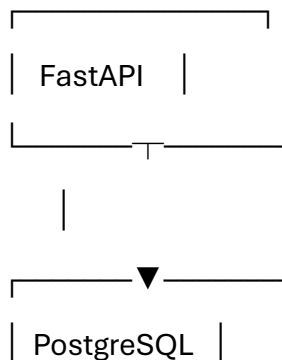
Stage 10 — Dockerize Everything

Goal

Make the application reproducible.

Architecture:

Docker Compose



|

|

| Streamlit |

|

Learn

- Dockerfile
- images
- containers
- volumes
- networks
- environment variables
- Docker Compose

Milestone 10

On a fresh machine:

`docker compose up`

and the application works.

Stage 11 — Deploy to the Cloud

Goal

Put the project somewhere accessible outside your laptop.

Cloud

Pick **AWS** and don't try to learn every AWS service.

Focus on:

- EC2
- RDS

- S3
- IAM
- CloudWatch

Deployment

Internet

↓

AWS

↓

Application

↓

PostgreSQL

Core decision

Understand **why each service exists**, rather than memorizing AWS features.

Milestone 11 — Deployment

- Application accessible remotely
- PostgreSQL hosted properly
- Secrets aren't hardcoded
- Logs available
- Basic monitoring
- Can deploy a new version

Stage 12 — Break It on Purpose

This is particularly important given the internship advice you received.

Goal

Learn troubleshooting rather than merely deployment.

Intentionally break:

database password

database connection

environment variables

SQL query

API input

container configuration

database availability

Then diagnose them.

Learn to use

- application logs
- PostgreSQL logs
- Docker logs
- HTTP status codes
- Linux commands
- CloudWatch

Milestone 12 — Troubleshooting

Given a broken application, you can systematically go:

What failed?

↓

Where did it fail?

↓

What do the logs say?

↓

Can I reproduce it?

↓

What's the root cause?

↓

Fix

↓

Verify

That's a very valuable milestone.

Stage 13 — Testing & Reliability

Goal

Make it a real software/data project rather than a demo.

Add

Unit tests

ETL functions

analytics functions

ML preprocessing

API logic

Data tests

No duplicate IDs

No invalid foreign keys

Valid dates

Valid ratings

Valid numerical ranges

API tests

valid request

invalid request

missing movie

database failure

Stack

- pytest
- FastAPI TestClient
- SQL/database testing

Milestone 13

You can intentionally introduce a bug and have tests detect it.

Stage 14 — Documentation & Finalization

Goal

Make the project understandable to an interviewer without you standing beside it.

Your README should contain:

1. What is this?
2. Why did I build it?
3. Architecture
4. Dataset
5. Database design
6. ETL
7. Analytics
8. Statistics
9. ML
10. API
11. Dashboard
12. Deployment
13. Testing
14. Troubleshooting
15. Limitations

Include:

- architecture diagram
- database ER diagram
- screenshots
- example SQL queries
- example API calls
- ML results
- deployment architecture
- important design decisions

Milestone 14 — Portfolio Ready

Someone can open your GitHub repository and understand:

what you built → why you built it → how it works → why you made your technical decisions → what you learned → what its limitations are.

Final Tech Stack

Don't add technologies just for the sake of having a huge stack.

Core

Python

SQL

PostgreSQL

Git/GitHub

Linux

Data

pandas

NumPy

SciPy

statsmodels

ML

scikit-learn

Application

FastAPI

Pydantic

SQLAlchemy

Streamlit

Infrastructure

Docker

Docker Compose

AWS

Testing

pytest

That's enough.

The Milestone Checklist

Keep this somewhere and **don't move to the next major stage until the previous one is genuinely working.**

- ☐ M0 Project scope defined
- ☐ M1 Raw data completely understood
- ☐ M2 Reproducible ETL pipeline
- ☐ M3 Proper PostgreSQL database
- ☐ M4 Strong SQL analytics
- ☐ M5 Statistical analysis + defensible conclusions
- ☐ M6 Revenue prediction + recommendation system

- ☐ M7 Reusable analytics layer
- ☐ M8 Working FastAPI
- ☐ M9 Working dashboard
- ☐ M10 Dockerized application
- ☐ M11 Cloud deployment
- ☐ M12 Troubleshooting practice
- ☐ M13 Automated tests
- ☐ M14 Documentation + portfolio ready

Most important progression

If university gets hectic, **do not try to maintain equal effort across everything.**

Your priority order should be:

SQL

↓

PostgreSQL

↓

Python/Data

↓

Statistics

↓

ML

↓

APIs

↓

Docker

↓

AWS



Testing/Reliability

And the key principle for the whole project:

Don't build technologies. Build one functioning system, and use each technology because the system requires it.

That will keep the project from turning into a giant collection of disconnected tutorials.